ANÁLISE COMPARATIVA DE ALGORITMOS DE ORDENAÇÃO: QUICK, MERGE, RADIX E HEAP SORT

Matheus Beuren
Disciplina de Pesquisa e Ordenação
matheusbeuren@gmail.com

Resumo

Este trabalho apresenta os resultados obtidos e a comparação de quatro algoritmos de ordenação, entre eles estão o Quick Sort, Merge Sort, Radix Sort e Heap Sort. Foram aplicados em arquivos de números inteiros com tamanhos entre 1 milhão e 2 milhões de registros, em três cenários de entrada (ordenado, invertido e randômico). Os algoritmos foram implementados em Assembly, uma linguagem de mais baixo nível para acelerar os processos, foram acionados por um menu principal escrito em Python, que também gera os arquivos de teste, e testados através da interface web usando o navegador Chrome. Os resultados mostram que os quatro algoritmos, todos O(n log n) na prática, têm tempo de execução muito baixo mesmo em arquivos grandes, ficando quase todos na casa dos milissegundos mesmo com 2 milhões de registros.

Abstract

This work presents the results obtained and the comparison of four sorting algorithms, namely Quick Sort, Merge Sort, Radix Sort and Heap Sort. They were applied to files of integer numbers with sizes ranging between 1 million and 2 million records, across three input scenarios (sorted, reversed and random). The algorithms were implemented in Assembly (NASM), a lower level language chosen to speed up the processes, triggered by a main menu written in Python, which also generates the test files, and tested through the web interface using the Chrome browser. The results show that the four algorithms, all O(n log n) in practice, have very low execution time even for large files, staying mostly in the millisecond range even with 2 million records.

1 INTRODUÇÃO

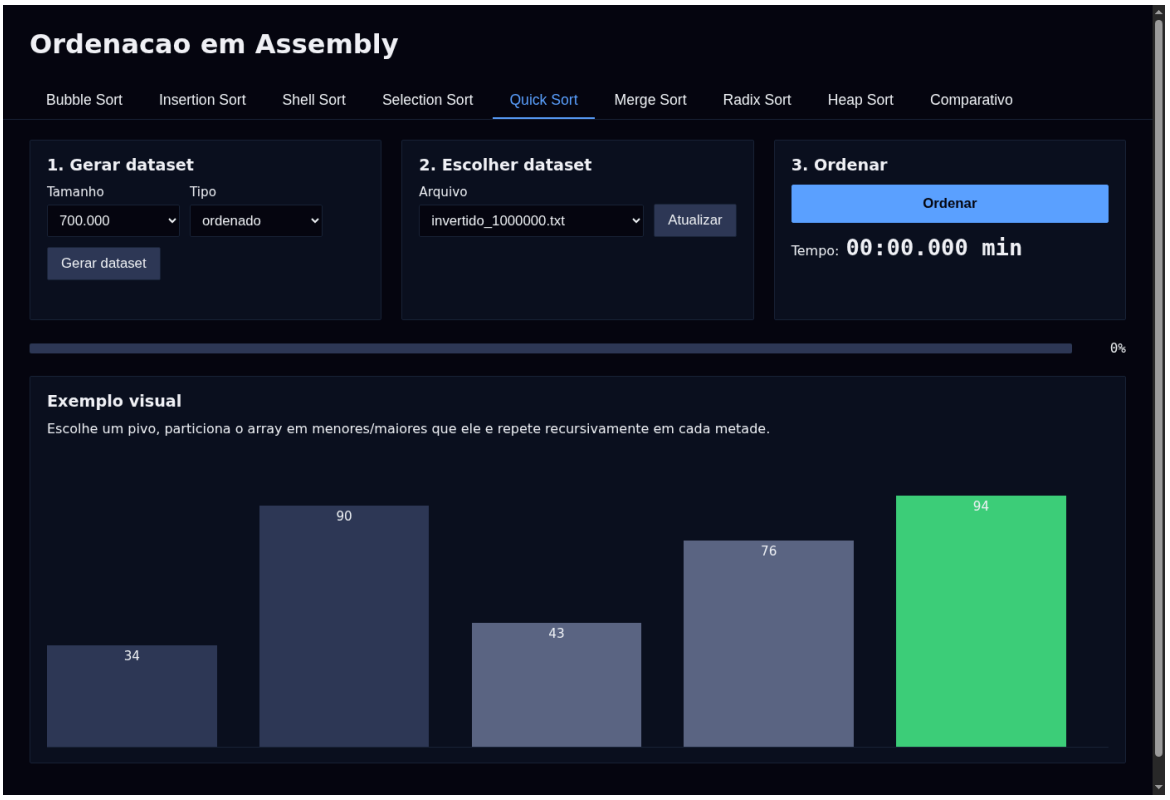
Ordenar dados é uma das tarefas mais feitas na computação, e a escolha do algoritmo certo muda muito o custo operacional de um sistema quando o volume de dados cresce. Dado um arquivo com milhões de registros, qual algoritmo resolve a ordenação em menor tempo, e para isso foram implementados quatro algoritmos, o Quick Sort, Merge Sort, Radix Sort e Heap Sort, de modo a comparar o tempo real de execução de cada um, aplicando-os a arquivos de 1.000.000 a 2.000.000 de registros.

Diferente dos algoritmos quadráticos avaliados anteriormente (Bubble, Insertion e Selection Sort), os quatro algoritmos deste trabalho são conhecidos por terem complexidade O(n log n), o que permite testar tamanhos bem maiores de arquivo sem o tempo de execução explodir.

2 METODOLOGIA

Para início dos testes, foi ampliado o mesmo programa em Python usado no trabalho anterior, que cria datasets, roda algoritmos e visualiza os resultados, de maneira que fosse de melhor entendimento dos algoritmos. A página pode ser vista na Figura 1.

Figura 1: Interface web, aba Quick Sort



Fonte: Autoria própria (2026)

Em vista de comparar diferentes algoritmos de ordenação, foram utilizados quatro algoritmos diferentes, como relatado na seção de introdução. Mas para mostrar um pouco de como o algoritmo influencia no tempo é utilizada a notação de Big O, que avalia o crescimento do tempo de execução e do uso de memória conforme o tamanho dos dados de entrada aumenta (CHIVERS; SLEIGHTHOLME, 2015). A Tabela 1 mostra como os algoritmos se descrevem nesta notação.

Tabela 1: Algoritmo x Big O

Algoritmo	Big O (pior caso)	Descrição
Quick Sort	$O(n^2)$	Particiona o array em torno de um pivô, recursivamente. Pivô de mediana entre três elementos evita o pior caso na prática.
Merge Sort	O(n log n)	Divide o array ao meio recursivamente e depois intercala as metades já ordenadas.

Algoritmo	Big O (pior caso)	Descrição
Heap Sort	O(n log n)	Monta um heap máximo e retira repetidamente o maior elemento, colocando-o na posição final.
Radix Sort	O(n × k)	Ordena por contagem, dígito a dígito, sem comparar os números entre si diretamente. k é o número de dígitos.

Fonte:Autoria própria(2026)

Todas as rotinas de ordenação foram escritas em Assembly x86-64, compiladas como biblioteca compartilhada e acessadas pelo menu e pela interface web via ctypes, o que garante que o tempo medido reflete o código de ordenação em si. Cada execução registra algoritmo, tipo de entrada, tamanho e tempo em segundos numa tabela histórica separada, que é a fonte dos dados deste relatório.

3 RESULTADOS

As Tabelas 2 a 5 mostram o tempo de execução de cada algoritmo, para os seis tamanhos de arquivo e os três tipos de entrada.

Tabela 2: Tempos de execução do Quick Sort

Tamanho	Ordenado	Invertido	Randômico
1.000.000	11,70 ms	33,20 ms	69,90 ms
1.150.000	17,64 ms	38,13 ms	81,66 ms
1.300.000	15,80 ms	44,36 ms	92,46 ms
1.350.000	17,51 ms	45,56 ms	95,69 ms
1.500.000	22,75 ms	51,22 ms	118,06 ms
2.000.000	30,24 ms	71,23 ms	147,54 ms

Fonte: Autoria própria(2026)

Tabela 3: Tempos de execução do Merge Sort

Tamanho	Ordenado	Invertido	Randômico
1.000.000	23,29 ms	22,65 ms	96,51 ms
1.150.000	26,83 ms	24,75 ms	110,21 ms
1.300.000	31,11 ms	27,55 ms	124,68 ms
1.350.000	34,51 ms	29,25 ms	136,10 ms
1.500.000	34,71 ms	32,74 ms	147,39 ms
2.000.000	46,74 ms	52,30 ms	200,48 ms

Fonte: Autoria própria(2026)

Tabela 4: Tempos de execução do Radix Sort

Tamanho	Ordenado	Invertido	Randômico
1.000.000	22,77 ms	19,76 ms	9,14 ms
1.150.000	30,77 ms	22,10 ms	10,63 ms
1.300.000	26,25 ms	27,53 ms	19,96 ms
1.350.000	34,76 ms	35,28 ms	15,40 ms
1.500.000	28,51 ms	28,12 ms	17,41 ms
2.000.000	43,76 ms	41,31 ms	27,48 ms

Fonte: Autoria própria(2026)

Tabela 5: Tempos de execução do Heap Sort

Tamanho	Ordenado	Invertido	Randômico
1.000.000	61,80 ms	67,29 ms	127,76 ms
1.150.000	72,21 ms	79,16 ms	152,10 ms
1.300.000	82,71 ms	88,67 ms	173,81 ms
1.350.000	85,36 ms	92,08 ms	182,69 ms

Tamanho	Ordenado	Invertido	Randômico
1.500.000	95,74 ms	102,41 ms	207,44 ms
2.000.000	129,09 ms	139,50 ms	279,25 ms

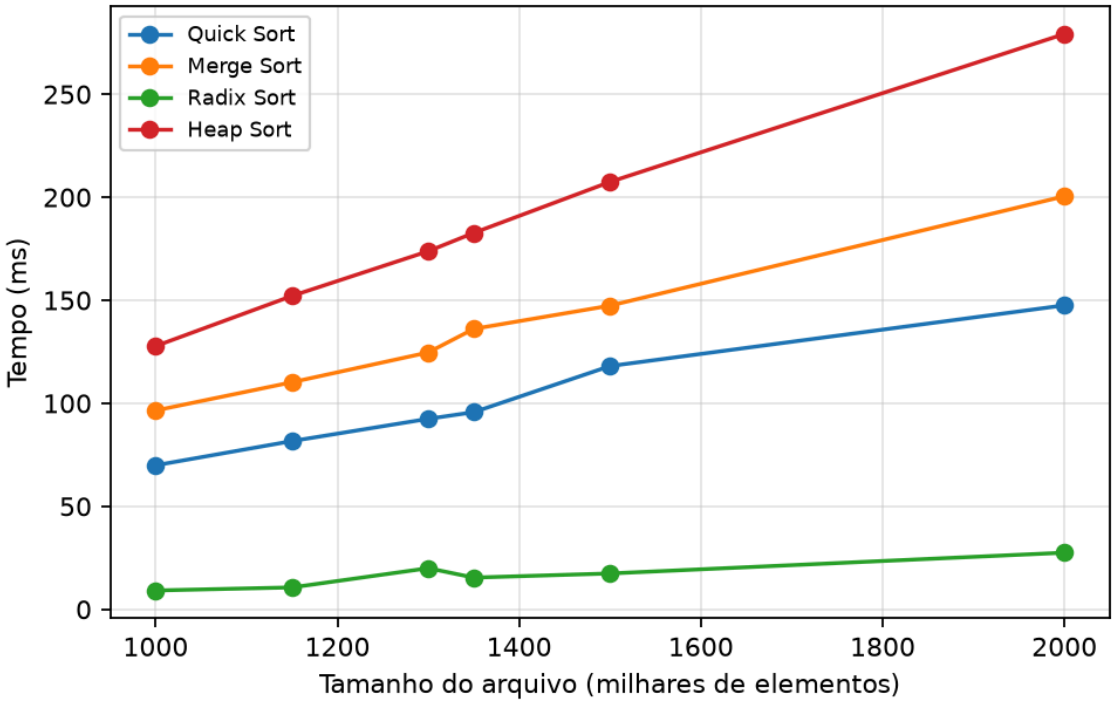
Fonte: Autoria própria(2026)

Olhando os resultados obtidos, fica claro que os quatro algoritmos são muito mais rápidos que os algoritmos quadráticos testados no trabalho anterior. Mesmo com 2.000.000 de registros, nenhum dos quatro passa de 0,3 segundos. Radix Sort é o mais rápido na maioria dos casos, principalmente na entrada randômica, porque não depende de comparações entre os números. Quick Sort e Merge Sort ficam bem próximos um do outro, com Quick Sort levemente mais rápido na entrada ordenada e invertida. Heap Sort é o mais lento dos quatro, embora ainda assim muito rápido, porque o acesso aos elementos do heap pula bastante na memória, o que custa mais na prática mesmo com a mesma complexidade assintótica dos outros.

4 GRÁFICOS COMPARATIVOS

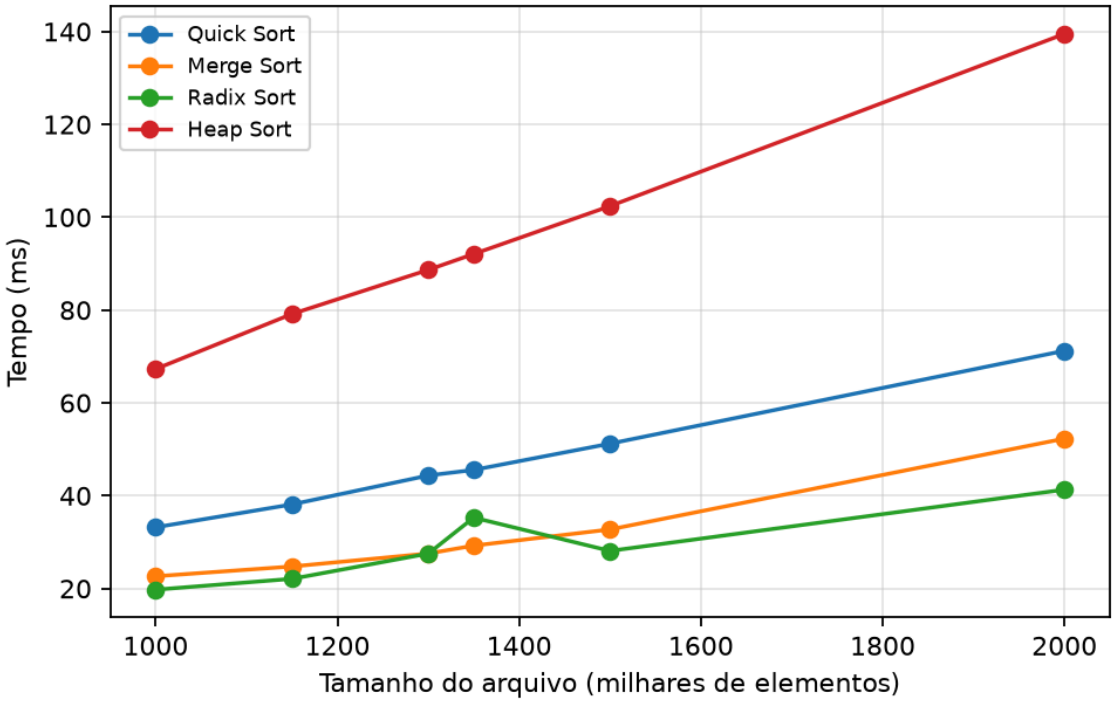
Os Gráficos 1 a 3 comparam os quatro algoritmos na entrada randômica, invertida e ordenada.

Gráfico 1: Entrada randômica



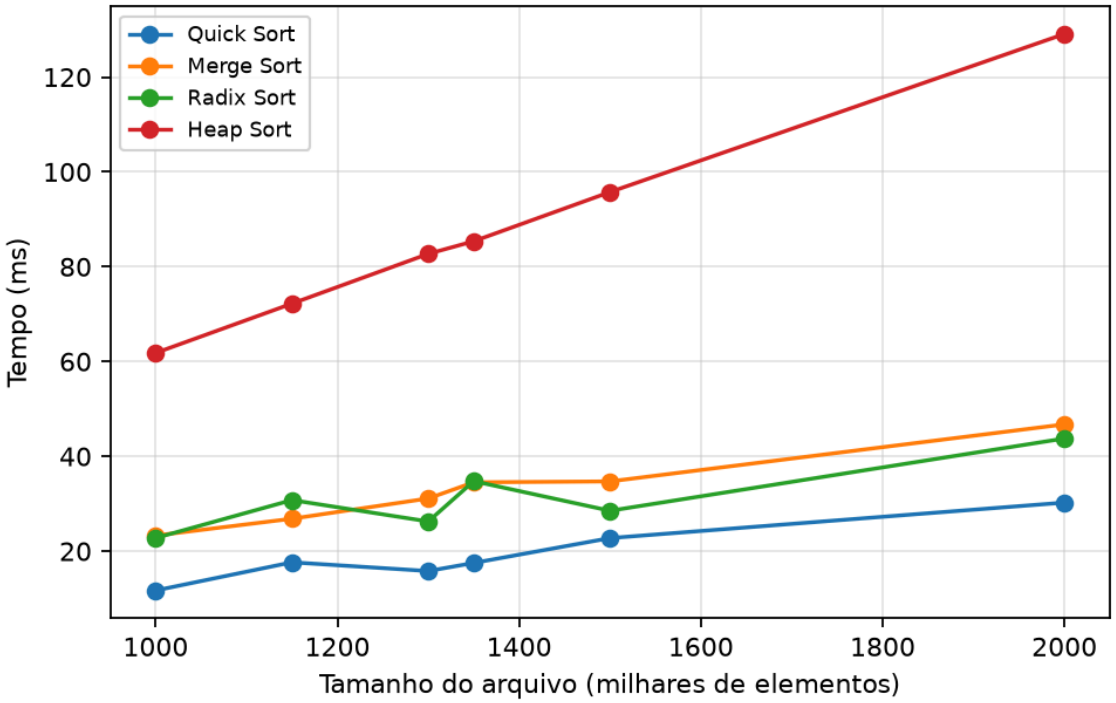
Fonte: Autoria própria(2026)

Gráfico 2: Entrada invertida



Fonte: Autoria própria (2026).

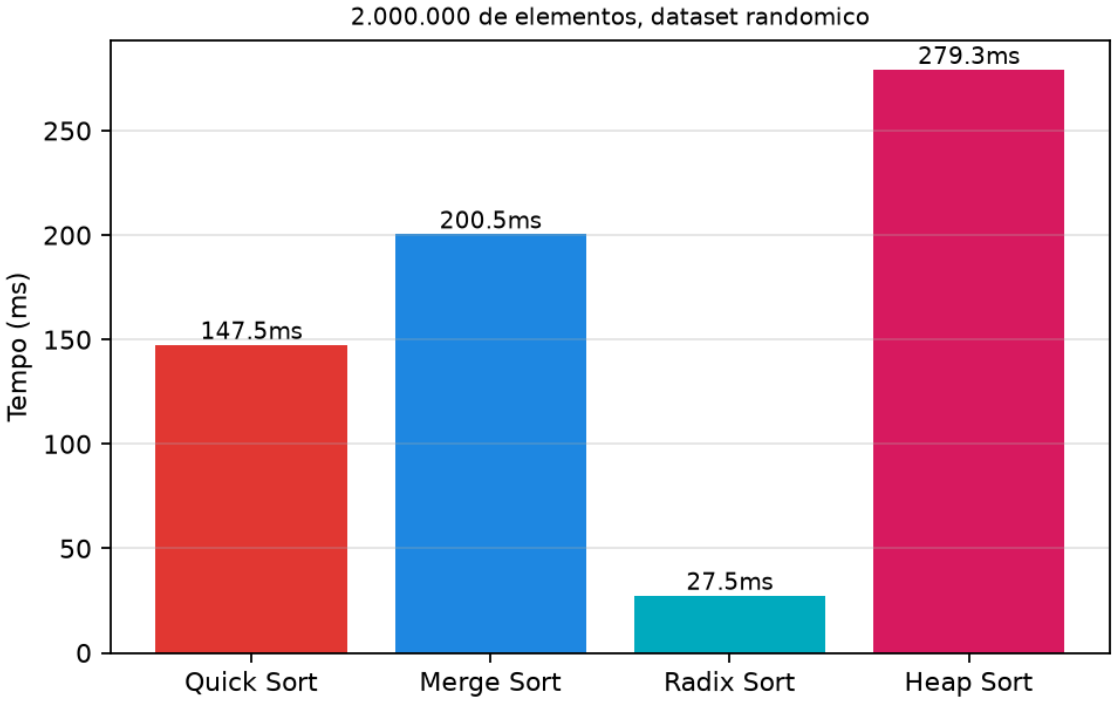
Gráfico 3: Entrada ordenada



Fonte: Autoria própria (2026).

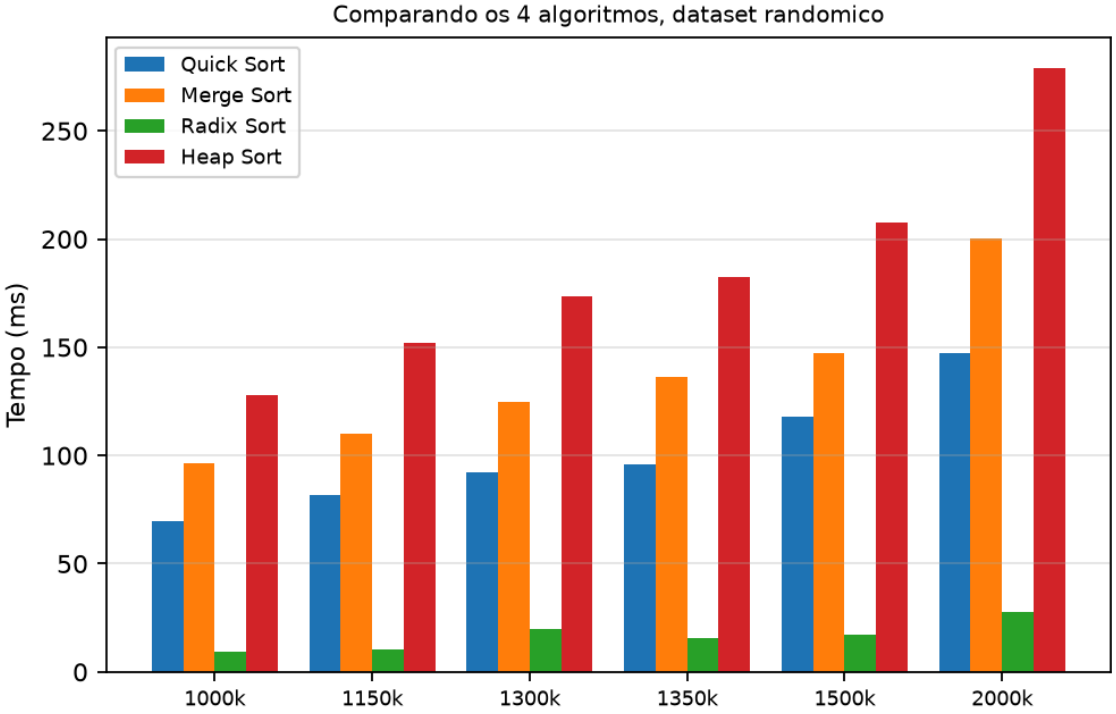
A seguir, o Gráfico 4 mostra a diferença dos algoritmos com entradas de 2.000.000 de dados randômicos.

Gráfico 4: Algoritmos x 2.000.000 de dados



Fonte: Autoria própria (2026).

Gráfico 5: Gráfico de barras agrupadas



Fonte: Autoria própria (2026).

5 DISCUSSÃO

Os resultados batem com o que a teoria de complexidade Big O mostra. Merge Sort e Heap Sort são O(n log n) garantido, até no pior caso. Quick Sort é $O(n^2)$ no pior caso clássico, mas a implementação usada neste trabalho escolhe o pivô pela mediana entre três elementos e sempre utiliza recursão no lado menor da partição, o que evita esse pior caso mesmo em entradas ordenadas ou invertidas, ficando O(n log n) na prática. Radix Sort nem compara os números entre si, então sua complexidade O(n × k) não depende da ordem inicial dos dados, só do número de dígitos.

Na prática, os quatro algoritmos deste trabalho são boas escolhas para arquivos grandes, ao contrário dos algoritmos quadráticos do trabalho anterior. Radix Sort se destaca quando os dados são números inteiros dentro de uma faixa conhecida, como é o caso aqui. Quick Sort e Merge Sort são mais genéricos e funcionam bem para qualquer tipo de dado comparável. Heap Sort tem a vantagem de ordenar em lugar sem precisar de memória extra, mas paga um preço em desempenho por causa do acesso espalhado na memória.

6 CONCLUSÃO

Este trabalho implementou e comparou quatro algoritmos de ordenação sobre arquivos de até 2.000.000 de dados, em três cenários de entrada, rodando todas as combinações através da interface web pelo navegador Chrome. Os testes confirmaram na prática que Quick Sort, Merge Sort, Radix Sort e Heap Sort são muito mais eficientes que os algoritmos quadráticos avaliados anteriormente, com tempos de execução na casa dos milissegundos mesmo nos maiores arquivos testados. Radix Sort se mostrou o mais rápido no geral, seguido por Quick Sort, Merge Sort e Heap Sort.

REFERÊNCIAS

CHIVERS, Ian; SLEIGHTHOLME, Jane. An introduction to Algorithms and the Big O Notation. In: Introduction to Programming with Fortran: With Coverage of Fortran 90, 95, 2003, 2008 and 77. Cham: Springer International Publishing, 2015. p. 359-364.